Deep Learning Based Fall Detection System

Talha Aslam
Universiti Kuala Lumpur, British Malaysian Institue, Malaysia
talhaaslamee@gmail.com

Aizat Faiz Ramli
University Kuala Lumpur, British Malaysian Institute, Malaysia
aizatfaiz@unikl.edu.my

Muhammad Noor Nordin
Universiti Kuala Lumpur, British Malaysian Institute, Malaysia
muhammadnoor@unikl.edu.my

Faezah Binti Harun
University Kuala Lumpur, British Malaysian Institute, Malaysia
faezah@unikl.edu.my

Kushsairy Abdul Kadir
University Kuala Lumpur, British Malaysian Institute, Malaysia
kushsairy@unikl.edu.my

2023 IEEE 9th International Conference on Smart Instrumentation, Measurement and Applications (ICSIMA) | 979-8-3503-4338-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICSIMA59853.2023.10373539

***Abstract*—A number of organizations, both medical and non-medical, have adopted assistive technology, such as medical wireless wristbands and fall detection devices. Most research on fall detection using deep learning techniques are based on image data sets, which are not readily available. So, researchers have focused on making a deep learning-based classifier that takes numerical input of sensors data and perform effectively, so that medical costs can be reduced. Deep learning-based fall detection is an assistive device to detect fall particularly for elderly people which can significantly reduce the mortality rate from falls by providing a response as part of a medical system. This paper developed a deep learning methodology to increase classifier accuracy using numerical data for fall detection. With the help of Deep Learning-Artificial Neural Network (DL-ANN), accelerometer, gyroscope, and infrared dataset are feed into DL-ANN consisting of convolution layer, pooling layer, and fully connected layer containing hidden layers to improve the classification accuracy. The DL-ANN was implemented on Jupyter Notebook with the help of a Python library for this Artificial Neural Network approach. This research also employs framework such as NumPy, Pandas, Scikit-Learn, Matplotlib, and Tensor Flow libraries. Moreover, the accuracy of the DL-ANN classifier was tested on eight optimizers with the help of different Keras layers in sequential model. The adaptive moment estimation (ADAM) algorithm performed the best in terms of loss while training the model. Deep Learning-Artificial Neural Network method helped to improve the classification accuracy of the wearable fall detection devices by up to 97%. This was done by training the data with a lot of computing power. Also, if falls could be reliably detected using a deep learning-based classifier like ANN, which got a 97% accuracy, 96% precision, 97% recall, and 96% F1-score rate in this paper, it would pave the way for the development of a fall detection solution that would work for lifelong.**

Keywords—Fall Detection, Deep Learning Artificial Neural Network, Convolution Neural Network.

I. Introduction

According to the World Health Organization [1], more than a third of persons 65 and over fall each year. For people above the age of 70, the ratio jumps to 32-42 percent. There is an increased danger of falling as because this analysis has gotten clumsier, it has grown more outdated. As a result, more persons are now harmed as a result of an elderly person falling. In the wearable based fall detection database [2], accelerometers and gyroscopes sensors built into smart phones are used. A form of machine learning known as "deep learning" is utilized in the training and decision-making processes that mimic human fall detection. Deep learning models has extensively studied over the past few decades in order to categories data and make human predictions [3]. Over the past few decades, fall detection and prediction systems have improved significantly, and the incidence of false alarms has dropped. When it comes to publications on fall detection, the top 25 nations in the world are depicted in Fig. 1 [4]. The United States supplies the majority of publications, with the top three providers being England, China, and Germany. The number of people above the age of 65 has increased in industrialized countries owed to better active conditions and better medical care.

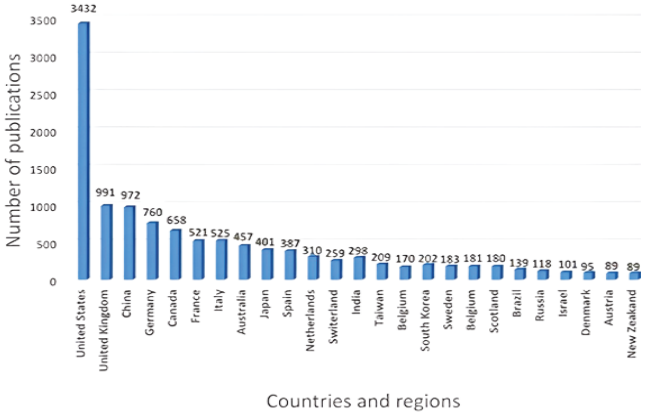


Fig. 1. Fall Detection World Map [4]

The analytical procedure for human fall detection is highly challenging, laborious and strongly dependent on individual perceptions and time restrictions. Although application of Artificial Neural Network for fall detection has been extensively investigated, there has been limited research investigating the deep learning neural networks classification performance in identifying human fall detection based on numeric dataset. Human fall data set may consist a large number of samples obtained from multiple sensors. Little studies have been done on the various deep learning neural networks optimizers in training and classifying performance for human fall detection. In this research paper, main objectives are to develop deep learning neural networks for human fall detection based on readings from inertial measurement unit (IMU) and gyroscope sensors. Additionally, researcher aims to investigate the training duration and classification performance of various deep learning neural networks using eight different optimizers.

II. Literature Review

A. Multi Model Human Fall Detection Techniques

An eigenspace-based neural network approach [5], used image data for detecting falls and figuring out the time of motion images. When motion pictures and the eigenspace technique are used together, they provide important information on human daily activities. These tests show that neural networks are a good way to detect human movements. It takes a dataset of ten different normal, abnormal, and unusual ways that people move in the ADL. The classification

979-8-3503-4338-0/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: VIT University. Downloaded on October 25,2026 at 17:05:18 UTC from IEEE Xplore. Restrictions apply.

experiment achieved accuracy rate of 91% which indicates satisfactory performance. In this paper [6], the decision tree method was used to detect the fall using tri-axial gyroscope sensors. This framework contains various activities such as standing, bending, sitting, and lying by using two tri-axial accelerometer sensors in different parts of the body. With accelerometers and gyroscopes, this algorithm reduces both false positives and false negatives in confusion matrix, making it better to detecting falls. Also, this solution has a low cost of computation and responds in real time. In [7] , image data were taken from a video recording of a person falling, as well as a shape projection on the x, y axis to recognize fall behaviors. An alarm will be triggered whenever a fall is predicted. This classifier actually based on time-frequency analysis and a deep learning network. In the fall dataset [8], they suggested a fall detection algorithm that examined various fall behaviors by adjusting the velocity and acceleration values of the sensor channels. Additionally, they used ANN to train the model using instances of labelled falling and non-falling activities. Paper [9] presents a method for automatic video fall detection. Indoor and outdoor videos showing different types of falls were recorded. For fall detection, they extract features from a gradient-distributed XY plane. The fall detection classifier was 100% accurate in experiments. Omni-Camera Images [10] featured different activity data to estimate statistically the threshold for fall detection at various height and weight ranges. In conclusion, the experimental results demonstrated that the success rates of fall detection are 79.8% and 68%, respectively, when personal information is included. A threshold-based fall detection algorithm using sensors in smart phone is presented in [11]. Specifically, accelerometers and gyroscopes sensors were used to detect falls. By using this methodology, if classifier detects fall, family members will receive an automatic message informing the status. Experiments used recorded 93.3% fall detection accuracy in activities daily living. To improve the accuracy of a smartphone's fall detection [12] result, the researchers in [12] used K-nearest neighbor KNN to classify accelerometer and a gyroscope sensors data to detect the presence of fall. The suggested system recorded a 99% accuracy rate in ADL falls, as determined by the K-nearest neighbor (KNN) method.

B. Benchmark Fall Data Base

In a benchmark paper [13], the authors tested several machine learning algorithms, namely support vector machine (SVM), K-nearest neighbor (KNN) and Neural Network (NN) for fall detection using Multimodal Human Action Database (MHAD). SVM classifier were shown to have accuracy of 90% with 95% specificity, whereas KNN specificity was 99% with accuracy about 92%. The neural network has accuracy of 95.49% and precision of 73% [13]. Although, various researchers have demonstrated the application of Artificial Neural Networks and Deep Learning to detect falls based on sensors data such as gyroscope and accelerometer [14]. Their model was trained to detect fall using data set consisting of limited range of activities such as walking, running, sitting, lying and standing. In this research, this paper aims to provide a robust way to detect fall using DL-ANN based on the Multimodal Human Action Database as presented in [13]. The data set consists of 6 different types of falls with 5 daily activities. This fall detection method improves the results in terms of accuracy, precision and F1 score shown in Section IV. Compared to Support Vector Machine, this DL-ANN scored increased 7% for accuracy, 32% for precision and 43% in F1 score. Compared to Random Forest, this model's accuracy increased by 2%, and its F1-score increased by 27%. When comparing the model to the Neural Network classifier, the accuracy increased by 5%.

III. Research Methodology

Researchers use a number of classifiers and medical approaches such as deep learning methods, utilizing data on patients' physical activities. An overview of the proposed model on different sensors data framework for deep artificial neural network. First, the accelerometer, gyroscope sensors data are obtained from standardized procedures. After receiving the data in csv files, the deep learning model is initialized with them and converted into array form. Next, the data is normalized before being convolved using the convolution layer with filter size and pooling layer with the configuration of stride and padding size. The model is then executed with good accuracy and precision by delivering inputs directly into a fully connected layer that has two hidden layers with ReLU activation functions. After each iteration, the artificial neural network model predicts falls for each other with new predictions and uses different sample selection methods to make sure training instances are accurate and varied. UP-Fall Detection dataset [13], which includes 11 activities and 3 trials per activity. All participants completed six common activities, daily living activities and five different types of falls. The data is divided into 70% for train and 30% for test size. To prevent overfitting the training data is further divided equally to 35% validation and training set. At *http://sites.google.com/up.edu.mx/har-up/*, the 812 GB consolidated dataset and the 171 GB feature dataset are available to the public. It was held at the Universidad Pan-American in Mexico City from June 18 to July 13, when the data was collected [13].

A. Deep Learning Artificial Neural Network

In Fig. 2, the numeric dataset which basically in array form, need to normalized first using min-max scaling. In convolution layer the authors set the filter or kernel size of (3x3) for purpose of convolving. After that Pooling layer specially set the values of padding and stride. Padding means to add zeros on the border to make information more informative. And stride is specially made for moving scale 1 step ahead on the matrix. Flattening layer connects the pooling layer with the fully connected layer in which the authors have 2 hidden layers worked with ReLU activation function. After whole process, output layers predict the output using sigmoid activation function whether patient is fall or not.

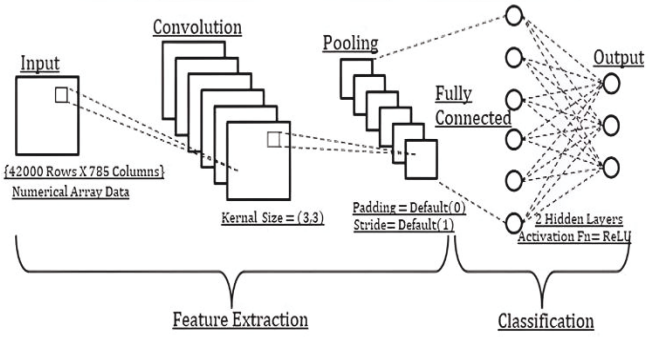


Fig. 2. Deep Learning Artificial Neural Network

The TensorFlow framework interface is implemented by keras. NumPy is the mother library to convert data into array

Authorized licensed use limited to: VIT University. Downloaded on October 25,2026 at 17:05:18 UTC from IEEE Xplore. Restrictions apply.

forms. Pandas used for data manipulation and analysis for python. Scikit-learn is a deep learning package for Python that can be used to import min-max scaler.

B. Model Work Flow

As can be seen in Fig. 3, the approach that is provided in this paper is comprised of three primary steps: dataset conversion using NumPy array, data pre-processing, and feature extraction and classification using a deep learning based Artificial Neural Network algorithm.

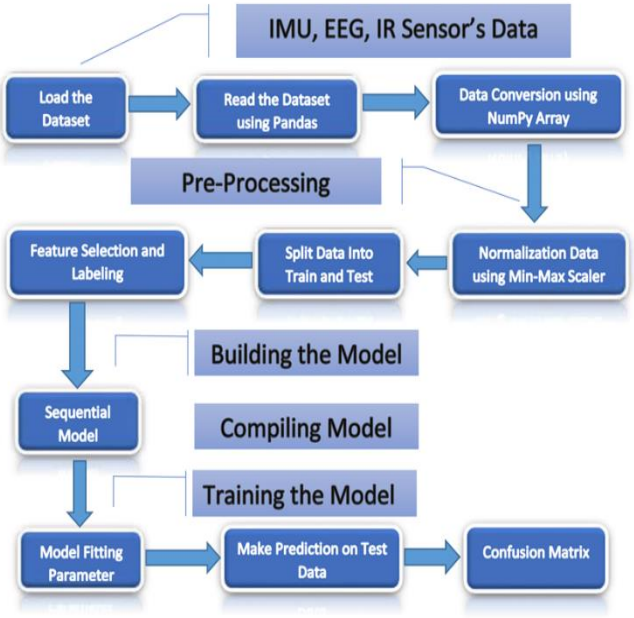


Fig. .3. Deep Learning Artificial Neural Network Workflow

In particular, the framework is broken down into three stages: Phase 1— preprocessing the acquired data into a form suitable for applying DL-ANN; In Phase 2— automatic feature extraction and learning for applying sequential model using Keras layers including hidden layers; and finally, Phase 3— determining the model accuracy using test dataset and classification. The authors decided to use mean values in order to make the dataset viable for applying an Artificial Neural Network model because the dataset contains a large number of anomalies, someplace extremely negative values and missing values, there the authors considered the mean and average values there.

C. Data Pre-Processing

For the data pre-processing, database needs to reshape and split into train and test values. This analysis considered X_train, X_test, Y_train and Y_test to the shapes that expects this model during train the model. Splitting the dataset is necessary for a fair assessment to predict accuracy. The data is divided into 70% for train and 30% for test size. To prevent overfitting the training data is further divided equally to 35% validation and training set. In most circumstances, dividing this dataset into three sections at random is sufficient. The training set is used to train or fit this model. Researcher used the training set to find the best the authorsights, or coefficients, for neural networks.

Researchers used validation set to fit the model using training set and check how well it works on the validation set for each hyperparameter setting. For a fair review of the final model, researcher needs the test set. It can't be used to fit or validate anything. To divide this data set into train, validation, and test with a test size ratio of 0.3. But validation actually provides us impartial valuation of the model fitted on training dataset though fine-tuning model hypermeters. So, second splitting with validation data ratio of 0.5. The formula used for second train, validation and test set used as following:

$$Second\ test_{size} = \left(\frac{1}{(1-test_{size})}\right) * validation_{size} \quad (1)$$

Deep learning-artificial neural network does not consider that extremist values. For that, this analysis will use Min-Max Scaler to minimize the values in the range of 0 and 1 and transform values of x into range of scale that the authors set in this model by using this formula.

$$x = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2)$$

D. Building The Model

Sequential is the model type that used in this network to create model in keras. That contain convolution layer, pooling layer and fully connected layer including hidden layers. Input layer contains sensors data which convolve throughout DL-ANN that will allows to build model layer by layer.

In this sequential model, there are 2 hidden layers with two-dimensional convolution with 32 neurons which actually deals as input with 2 dimensional metrices. Dense (32) is fully connected layer actually with 32 pooling sizes, also can adjust these numbers to be lower or higher depend upon the dataset size. At output layer has 1 node on should be same as no of classes, for every possible outcome. The Sequential model type is going to be the one that this analysis focus on developing for this application. Using the sequential strategy while building a model in Keras is the approach that is the least complicated to follow. It enables the authors to build a model in phases, adding layers successively as researchers go. This analysis make use of the 'add ()' method so that this model can have additional layers added to it. This structure is made up of three layers, the first two of which Dense (32) are hidden layers. These convolution layers, which will be in charge of handling the numeric values, will perform the necessary processing and then normalize it using min max scaler. Hidden layers consisting of convolution layer, ReLU, pooling layers as well as fully connected layer. The following is a breakdown of the number of nodes that can be found in each layer: 32 can be found in the first layer, and 32 can be found in the second layer. The size of the dataset that is being utilized can have an effect on the size of this amount, causing it to either be increased or decreased. As a result of the fact that both 32 nodes work well for us, this configuration will most likely remain the one that analysis use for the predictable future. A rectified function while designing neural network, Multiple hidden layers of neurons activated by RELU function. It is used for binary classification and calculating the probability values.

$$ReLU(x) = \max(0, x) \quad (3)$$

Sigmoid curve looks alike S shaper which converts independent variables of near infinite range into 0 and 1.

$$Sigmoid(x) = \sigma = \frac{1}{1 + e^{-x}} \quad (4)$$

Authorized licensed use limited to: VIT University. Downloaded on October 25,2026 at 17:05:18 UTC from IEEE Xplore. Restrictions apply.

E. Compiling the Model:

Compiling the model actually takes three parameters such as following: optimizer, loss and metrics. Optimizers are functions or algorithms that change the features of said neural network, such as with the number of the weights and the rate at which it learns new information. Eight optimizers including Stochastic Gradient Descent (SGD), Root Mean Square Propagation (RMSPROP). Adaptive Moment Estimation (ADAM), Adaptive Learning Rate Method (ADADELTA), Adaptive Gradient Algorithm (ADAGRAD), Nesterov-Accelrated Adaptive Moment Estimation (NADAM), ADAMAX and Follow The Moving Leader (FTRL) used for getting good accuracy.

F. Training the Model

The model was trained in this analysis using the fit function with the following parameters: (X_val), (Y_val), validation data, and number of epochs. In sequential model, the number of training samples used in one round is called the "batch size.", In this analysis, the batch size was set at 32 so that the model could be fit. Iteration and epoch values are the same when the batch size is the same as the whole dataset.

IV. Results and Discussion

The numeric dataset used for this DL-ANN model actually consist on 42000 rows and 756 columns. Dataset includes the fall detection sensors output values performing 11 different activities with 3 trials on each subject. So, The np. array () function accepts an inerrable and returns a NumPy array, which creates a new memory data structure. In this data base, 42000 rows and 785 columns available in array form. The pixel term used for this easiness to convert all sensors channel name into pixels up to 785 columns. This model taken first 700 columns as training and 41300 taken as testing purpose. This analysis predicts labels using features (the analysis use to predict labels) (the data analysis wants to predict). That explained us the features and label selection for both training and testing into shapes form.

A. Min-Max Scaling and Model Fit

The Minmax Scaler function reduces data to fit inside a specific range, which is commonly 0 to 1 shown in Fig. 4.

```
array([[0.11111111, 0.        , 0.        , ..., 0.        , 0.
        0.        ],
       [0.        , 0.        , 0.        , ..., 0.        , 0.
        0.        ],
       [0.11111111, 0.        , 0.        , ..., 0.        , 0.
        0.        ],
       ...,
       [0.77777778, 0.        , 0.        , ..., 0.        , 0.
        0.        ],
       [0.66666667, 0.        , 0.        , ..., 0.        , 0.
        0.        ],
       [1.        , 0.        , 0.        , ..., 0.        , 0.
        0.        ]])
```

Fig. 4 Min-Max Scaling

The model cannot be compiled without the following parameters: optimizer, loss, and metrics. This optimizer is in charge of determining the rate of learning.

B. Model Plot for SGD with 15 Epochs

One of the most popular charts for debugging neural networks is the loss curve during training, as shown in Figure 5. Notably, at the 14th epoch, the model achieved a remarkable loss of 0.00010.

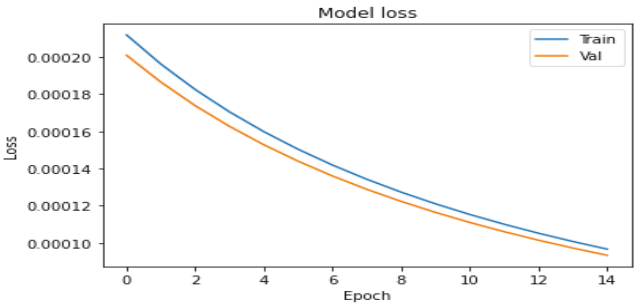


Fig. 5. Model Plot for SGD with 15 Epochs

C. Model Plot for RMSPROP with 15 Epochs

A loss curve during training is a fundamental tool for debugging neural networks, offering insights into the network's learning process and trajectory, as depicted in Figure 6. Notably, the model achieved a perfect loss of 0.040 at the 1st epoch, highlighting its rapid convergence.

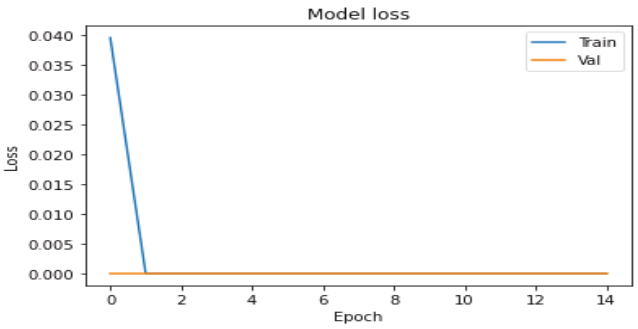


Fig. 6. Model Plot for RMSPROP with 15 Epochs

D. Model Plot for ADAM with 15 Epochs

Among the best optimizers, the loss curve helps diagnose neural network issues. Figure 7 shows that ADAM optimizer achieved an outstanding initial performance with a minimal loss of 0.035 at the 1st epoch.

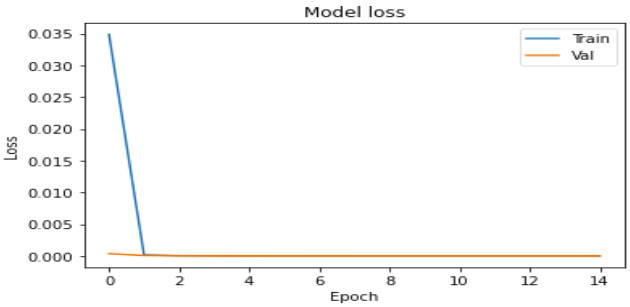


Fig. 7. Model Plot for ADAM with 15 Epochs

E. Model Plot for ADADELTA with 15 Epochs

The loss curve is a common tool for neural network analysis. In Figure 8, researchers observe ADADELTA's challenging start with a 0.7 loss at the beginning of training.

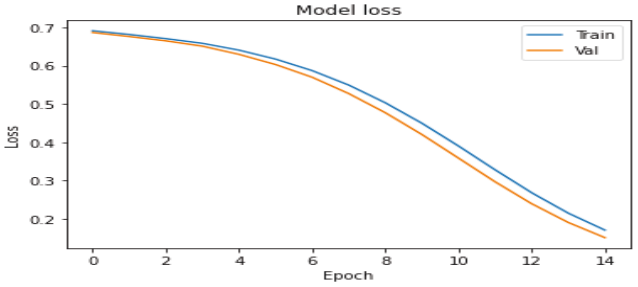


Fig. 8. Model Plot for ADADELTA with 15 Epochs

Authorized licensed use limited to: VIT University. Downloaded on October 25,2026 at 17:05:18 UTC from IEEE Xplore. Restrictions apply.

F. Model Plot for ADAGRAD with 15 Epochs

In Figure 9, it's evident that ADAGRAD requires more epochs to achieve a lower loss, as shown in the loss curve—a key tool for neural network analysis during training.

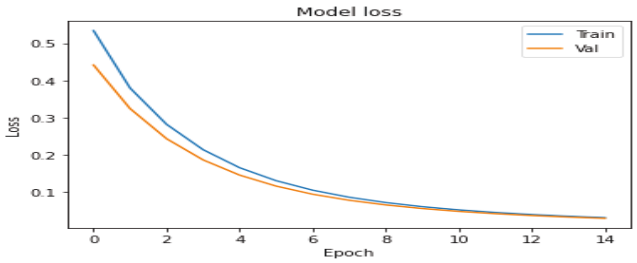


Fig. 9. Model Plot for ADAGRAD with 15 Epochs

G. Model Plot for ADAMAX with 15 Epochs

In Figure 10, researchers observed an initial achievement with a loss of 0.05 at the 1st epoch. The loss curve, a valuable tool for monitoring neural network training, sheds light on the network's learning journey.

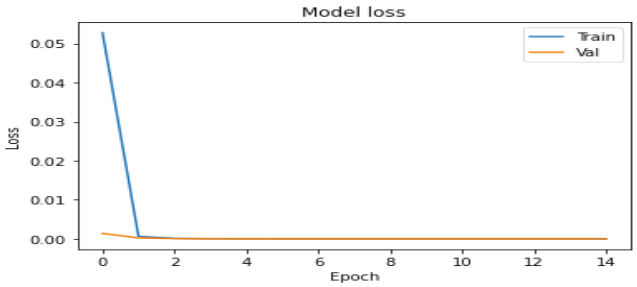


Fig. 10. Model Plot for ADAMAX with 15 Epochs

H. Model Plot for NADAM with 15 Epochs

Loss curves are used to debug neural networks. It shows the network's learning process and direction define in Fig. 11.

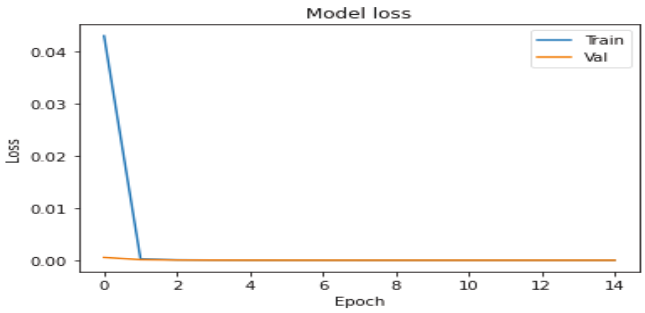


Fig. 11. Model Plot for NADAM with 15 Epochs

I. Model Plot for FTRL with 15 Epochs

Figure 12 highlights a significant milestone at the 14th epoch, where the loss reached 0.25. This loss curve is a crucial aid for understanding the neural network's training process and progress.

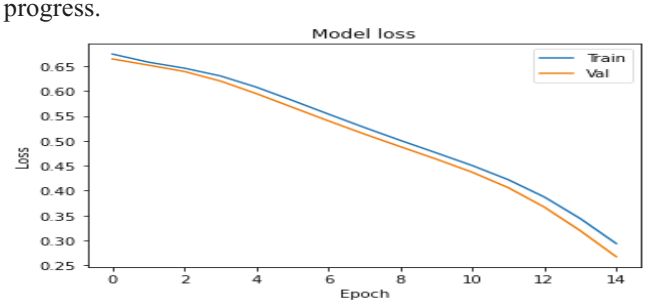


Fig. 12. Model Plot for FTRL with 15 Epochs

J. Evaluate Model Performance

Table I. shows the number of epochs needed to achieve the minimum loss performance for various optimizers to train the model. From the result, the authors can conclude that the ADAM optimizer provides the best balance in terms of loss performance and the training duration. Although, SGD algorithm produced least loss performance, it requires more training time at 14 epochs to match ADAM minimum loss. Adadelta, Adagrad and FTRL performed very poorly due to computation time, and unable to achieve less loss. RMSPROP and NADAM optimizers have similar performance to ADAM but generates slightly higher maximum loss performance. All optimizers plot showed graphically loss performance with maximum training duration of 15 epochs. The best optimizers are able to quickly determine weight values that produces the least amount of loss performance. Blue line represents the train and orange line represents the validation process.

Table I. Optimizers Results Comparison

No.	Optimizer	Max Loss	Min Loss	Epoch
1	SGD	0.00020	0.0001	14
2	RMSPROP	0.040	0.0001	1
3	ADAM	0.035	0.0001	1
4	ADADELTA	0.7	0.1	14
5	ADAGRAD	0.5	0.09	10
6	ADAMAX	0.05	0.0001	1
7	NADAM	0.04	0.0001	1
8	FTRL	0.65	0.25	14

K. Performance Evaluation with Confusion Matrix

This analysis also validates the DL-ANN over the testing data and performed results showed in Fig. 13.

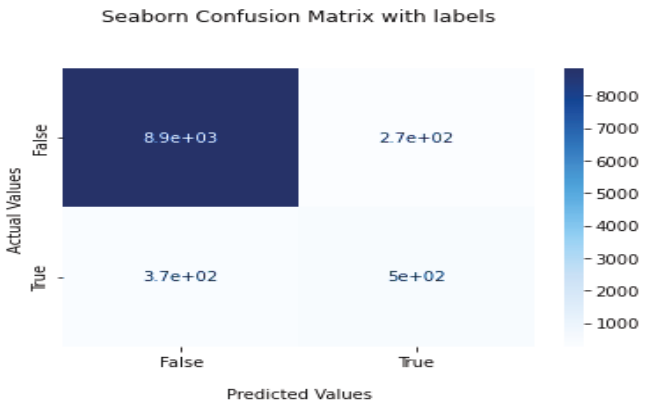


Fig. 13. Performance Evaluation with Confusion Matrix

Using Adaptive Moment Estimation (ADAM) optimizer as stated in Fig. 13, this seaborn confusion matrix showed the fall prediction, samples basically corresponding to fall patient which presents the positive class, and somewhere other corresponding to non-fall patient represents the negative class. It can be observed that model DL-ANN performed different attributes in term of accuracy, recall, F1- score and support, whether 0.0 presents the predicted values output of the training model and 1.0 represents the actual values output can see in Table II.

Authorized licensed use limited to: VIT University. Downloaded on October 25,2026 at 17:05:18 UTC from IEEE Xplore. Restrictions apply.

Table II. Performace Evaluation

	Precision	Recall	F1-score	Support
0.0	96%	97%	96%	9130
1.0	65%	57%	61%	870
Validation Accuracy	100%	94%	94%	10000

L. Comparison with Benchmark Paper

In benchmark paper, different classifier has been applied namely as: support vector machine (SVM), random forest (RF), K-nearest neighbor (KNN) in Table IIII [13]. Compared to SVM, this DL-ANN got 7% better at accuracy, 32% better at precision, and 43% better at F1-score. Compared to RF, this model's accuracy got 2% greater, its precision got 22% greater, and its F1-score went up 27%. By comparing the model to the KNN classifier, the accuracy has gone up by 5%. Table IV.3 clearly shows that this Deep Learning-Artificial Neural Network has improved both precision and F1-score by 25% and 36%, respectively.

Table III. Comparison Results with Benchmark Paper

Model	Accuracy	Precision	F1-score
SVM [13]	90%	64%	53%
RF [13]	95%	74%	69%
KNN [13]	92%	71%	60%
NN [13]	95%	72%	71%
DL-ANN	97%	96%	96%

V. CONCLUSION AND FUTURE WORKS

In conclusion, the implementation of a Deep Learning-Artificial Neural Network on a public dataset yielded impressive results, with a notable 97% accuracy, 96% precision, and a robust 96% F-1 score. Support Vector Machine (SVM) and Random Forest (RF) algorithms were considered for the task, but historical issues related to dataset balance limited their performance. Comparing the results, the DL-ANN demonstrated clear superiority, achieving a 7% increase in accuracy over SVM, a substantial 32% improvement in precision, and an impressive 43% boost in F1-score. In comparison to RF, the model displayed a noteworthy 2% accuracy enhancement, a remarkable 22% increase in precision, and a substantial 27% rise in F1-score. Additionally, the DL-ANN outperformed K-Nearest Neighbors (KNN) by 5% in terms of accuracy.

Recognizing the inherent complexity of deep learning networks for numerical data, future research will explore simplified methods for transmitting data in image format through convolution techniques. The incorporation of accelerometer and gyroscope sensor data in upcoming work aims to further enhance image resolution for improved results..

REFERENCES

[1] M. Montero-odasso *et al.*, "World guidelines for falls prevention and management for older adults : a global initiative," pp. 1–36, 2022.

[2] F. Luna-Perejón, M. J. Domínguez-Morales, and A. Civit-Balcells, "Wearable Fall Detector Using Recurrent Neural Networks.," *Sensors (Basel).*, vol. 19, no. 22, Nov. 2019, doi: 10.3390/s19224885.

[3] J. Wu, J. Wang, A. Zhan, and C. Wu, "Fall detection with cnn-casual lstm network," *Inf.*, vol. 12, no. 10, 2021, doi: 10.3390/info12100403.

[4] X. Wang, J. Ellul, and G. Azzopardi, "Elderly Fall Detection Systems: A Literature Survey," *Front. Robot. AI*, vol. 7, no. June, 2020, doi: 10.3389/frobt.2020.00071.

[5] H. Foroughi, A. Naseri, A. Saberi, and H. S. Yazdi, "An eigenspace-based approach for human Fall detection using integrated time motion image and Neural Network," *Int. Conf. Signal Process. Proceedings, ICSP*, no. November, pp. 1499–1503, 2008, doi: 10.1109/ICOSP.2008.4697417.

[6] K. Singh, A. Rajput, and S. Sharma, "Human fall detection using machine learning methods: A survey," *Int. J. Math. Eng. Manag. Sci.*, vol. 5, no. 1, pp. 161–180, 2020, doi: 10.33889/IJMEMS.2020.5.1.014.

[7] F. Ofli, R. Chaudhry, and G. Kurillo, "Berkeley Multimodal Human Action Database (MHAD)," *IEEE Work. Appl. Comput. Vis.*, pp. 53–60, 2013, [Online]. Available: http://tele-immersion.citris-uc.org/berkeley_mhad/

[8] J. Gutiérrez, V. Rodríguez, and S. Martin, "Comprehensive review of vision-based fall detection systems," *Sensors (Switzerland)*, vol. 21, no. 3, pp. 1–50, 2021, doi: 10.3390/s21030947.

[9] S. Nooruddin, M. M. Islam, F. A. Sharna, H. Alhetari, and M. N. Kabir, "Sensor-based fall detection systems: a review," *J. Ambient Intell. Humaniz. Comput.*, vol. 13, no. 5, pp. 2735–2751, 2022, doi: 10.1007/s12652-021-03248-z.

[10] S.-G. Miaou, P.-H. Sung, and C.-Y. Huang, "A Customized Human Fall Detection System Using Omni-Camera Images and Personal Information," in *1st Transdisciplinary Conference on Distributed Diagnosis and Home Healthcare, 2006. D2H2.*, 2006, pp. 39–42. doi: 10.1109/DDHH.2006.1624792.

[11] S. Usmani, A. Saboor, M. Haris, M. A. Khan, and H. Park, "Latest research trends in fall detection and prevention using machine learning: A systematic review," *Sensors*, vol. 21, no. 15, pp. 1–23, 2021, doi: 10.3390/s21155134.

[12] E. Casilari-Pérez and F. García-Lagos, "A comprehensive study on the use of artificial neural networks in wearable fall detection systems," *Expert Syst. Appl.*, vol. 138, p. 112811, 2019, doi: https://doi.org/10.1016/j.eswa.2019.07.028.

[13] L. Martínez-Villaseñor, H. Ponce, J. Brieva, E. Moya-Albor, J. Núñez-Martínez, and C. Peñafort-Asturiano, "UP-Fall Detection Dataset: A Multimodal Approach.," *Sensors (Basel).*, vol. 19, no. 9, Apr. 2019, doi: 10.3390/s19091988.

[14] A. Z. Rakhman, L. E. Nugroho, Widyawan, and Kurnianingsih, "Fall detection system using accelerometer and gyroscope based on smartphone," *2014 1st Int. Conf. Inf. Technol. Comput. Electr. Eng.*, pp. 99–104, 2014.

Authorized licensed use limited to: VIT University. Downloaded on October 25,2026 at 17:05:18 UTC from IEEE Xplore. Restrictions apply.